

Partie 1 : Méthode de paiement sécurisé

- 3. Exécution du finally : Garanti par la JVM, il s'exécute toujours (après try ou catch). Son but est de libérer les ressources ou fermer les processus (ex: connexions, fichiers), peu importe si une erreur survient ou non.
- 4. Erreur Métier vs Technique :
 - Métier (Solde insuffisant) : Violation d'une règle fonctionnelle (le système marche, mais le client n'a pas les fonds).
 - Technique (Montant ≤ 0) : Violation d'une règle structurelle ou bug (données invalides ou mauvaise utilisation du système).

Partie 2 : Gestion locale des exceptions

- 6. Exception Automatique vs Manuelle :
 - Automatique : Levée par la JVM face à une action impossible (ex: `Double.parseDouble("abc")` → `NumberFormatException`).
 - Manuelle : Déclenchée par le développeur via le mot-clé `throw` pour bloquer un comportement non souhaité (ex: montant négatif).

Partie 3 : Propagation des exceptions

- 9. Propagation et Comparaison :
 - Mécanisme : `lireMontant` n'attrape pas l'erreur mais la renvoie (`throws`) à la méthode appelante (`main`).
 - Comparaison : En Partie 2 (locale), l'erreur est gérée sur place, renvoie 0 et le code continue. En Partie 3 (propagation), la méthode s'interrompt et délègue la décision au `main`.
- 10. Absence de gestion dans le `main` : L'exception remonte à la JVM, le programme crash (arrêt brutal) et affiche la pile d'exécution (*stack trace*) dans la console.
- 11. Pourquoi propager ? Quand la méthode ne dispose pas du contexte pour régler l'erreur (ex: un outil de calcul ne sait pas s'il faut afficher un pop-up ou écrire dans un fichier). Cela centralise la décision dans les couches supérieures .

Partie 4 : Lecture d'une transaction

- 14. Tableau trop court : Si l'index $\geq$ la taille du tableau, la JVM lève automatiquement une `ArrayIndexOutOfBoundsException`, stoppant l'accès car l'indice est invalide.